Project Report On Modern Application Development-II

Author

Name: Hanzala Tafzeel
Roll No: 23f2004513
Email: 23f2004513@ds.study.iitm.ac.in

Description

A smart online quiz website built with two separate parts - a backend system that handles data and a frontend that users see. Admin can create subjects, add chapters, make quizzes, and manage students through an easy-to-use control panel. Students can sign up, take quizzes, and see their scores and progress. The system includes secure login, timed quizzes, data downloads, and automatic emails.

Technologies used

- Backend Framework: Flask REST API, JWT Authentication, Flask-SQL Alchemy, Celery, Flask-Mail, Flask-Caching, Redis.
- Frontend: Vue.js, HTML, CSS, Bootstrap.
- Database: SQLite for storing user data, requests, and roles, etc.

DB Schema Design - Reasons Behind Design

The quiz platform uses a normalized 6-table database structure: User (authentication with role-based access), Subject (content organization), Chapter (subject subdivisions), Quiz (scheduled assessments with timing controls), Question (4-option MCQs with individual scoring), and supporting tables QuizAttempt and QuestionResponse for tracking user performance. The design employs foreign key constraints for referential integrity, cascade deletes for data consistency, unique constraints on critical fields (email, quiz titles), and soft delete mechanisms via boolean flags. Key features include automatic marks calculation, flexible quiz scheduling, pre-calculated correctness scores for performance, and comprehensive audit trails through timestamp tracking, ensuring scalable and secure quiz management.

API Design

The API is created for a comprehensive quiz management system, supporting user authentication, user management, subject and chapter management, quiz and question management, quiz attempts and scoring, analytics, and utility operations (like notifications and CSV export). Key elements include robust REST endpoints for users (including admin/user roles), subjects, chapters, quizzes, questions, and attempts—secured by JWT authentication.

Architecture and Features

The project follows a modern decoupled architecture with separate backend and frontend applications:

Backend Structure (Flask REST API)

app.py - Initializes Flask application, configures JWT, database, Celery, and registers API routes, models.py - Defines SQLAlchemy models for User, Subject, Chapter, Quiz, Question, QuizAttempt, and QuestionResponse, routes.py - Contains all REST API endpoints organized by functionality (authentication, user management, quiz operations, analytics), config.py - Configuration settings for different environments (development, production), requirements.txt - Python dependencies (Flask, SQLAlchemy, JWT, Celery, etc.)

Frontend Structure (Vue.js SPA)

src/main.js - Vue application entry point with router and store configuration, src/components/ - Reusable Vue components (QuizCard, UserDashboard, AdminPanel), src/views/ - Page-level components (Login, QuizAttempt, Analytics, AdminDashboard), src/router/ - Vue Router configuration with authentication guards, src/store/ - Vuex state management for user data, quizzes, and authentication, src/assets/ - Static files (CSS, images, Bootstrap customizations), package.json - Node.js dependencies and build scripts

Key Architectural Features

- RESTful API Design - Complete separation of concerns with JSON-only communication
- JWT Authentication - Stateless token-based security with automatic header injection
- Component-Based UI - Modular Vue.js components for maintainability
- Responsive Design - Bootstrap 5 integration for mobile-first experience
- Asynchronous Processing - Celery integration for background tasks
- Caching Strategy - API response optimization for better performance

This architecture enables independent deployment, better scalability, and potential mobile app development using the same REST API.

Video

Click Here

ER Diagram

Click Here